

Learn MS SQL Server With Db Design Techniques

...



Mohd Manzoor Ahmed

(MCT | Founder Of www.ManzoorTheTrainer.com)



What Are We Going To Cover?

- Introduction To RDBMS
- Primary Key
- Unique Key
- Data Types
- Foreign Key
- Database Design Techniques
- T-SQL Select Query
- Joins
- Sub Queries
- Correlated Subqueries
- Stored Procedures
- Functions
- Views
- Transactions
- Some Admin Tasks
- Assignment
- Q & A

Prerequisite

- You should know how to use computer.
- Install MS SQL Server 2019 Express Edition
<https://www.microsoft.com/en-us/Download/details.aspx?id=101064>
<https://docs.microsoft.com/en-us/sql/ssms/download-sql-server-management-studio-ssms?view=sql-server-ver15>
- Operating System : Windows 10 , Windows 8, Windows 8.1, Windows Server 2012, Windows Server 2012 R2, Windows Server 2016
- Processor : Intel - compatible processor with a minimum speed of 1 GHz
- RAM : Minimum 512MB
- Hard Disk Space :4.2 GB of Disk Space

Note : Disable prevent saving changes that require table re-creation.

(Tools → Options → Designers : Uncheck)



What is RDBMS?

- A relational database consists of a collection of tables
- A table consists of rows and columns
- Each row represents a record
- Each column represents an attribute of the records contained in the table
- MS SQL Server, Oracle, DB2, MySQL, etc.,
- We interact through SQL.

EmpId	Name	Gender	Email
1	Ahmed	M	ahmed@gmail.com
2	Peter	M	peter@hotmail.com
3	Lilly	F	lilly@gmail.com
4	Tom	M	tom@gmail.com

Primary Key

Empld	Name	Gender	Email	Contact
1	Ahmed	M	ahmed@gmail.com	9876786545
2	Peter	M	peter@hotmail.com	NULL
3	Lilly	F	lilly@gmail.com	8798765643
4	Tom	M	tom@gmail.com	9454323454
5	Tom	M	tom_1@gmail.com	9878765434

Primary Key Vs Unique Key

- **Primary Key**
 - It cannot be Null.
 - It cannot have duplicate values.
 - A table can have one and only one primary key
- **Unique Key**
 - It can have one and only one Null.
 - It cannot have duplicate values.
 - A table can have more than one unique key

Composite Primary Key

StudentId	SubjectId	Marks
1	101	67
1	102	68
1	103	56
2	101	78
2	102	77
2	103	79
3	101	NULL
3	102	56
3	103	55

Get Started With MS SQL Server

- Create Database
- Create Table
- Create Primary Key
- Identity Column
- Set Default Values
- Create Unique Key
- Create Check Constraints

Demo

Let's see a demo on getting started with MS SQL Server

Data Types In MS SQL Server

- String
 - char(n) (Fixed Non - Unicode)
 - varchar(n) (Variable Non - Unicode)
 - nchar(n) (Fixed Unicode)
 - nvarchar(n) (Variable Unicode)
- Number
 - int, bigint, smallint, and tinyint
 - decimal and numeric
 - float and real
 - money and smallmoney

Data Types In MS SQL Server

- DateTime
 - date
 - time
 - datetime
 - datetime2
 - smalldatetime
- Uniqueidentifier
- Bit (0 or 1)
- For Ranges : https://www.w3schools.com/sql/sql_datatypes.asp

Foreign key

EmpId	Name	Gender	Email	Contact	Did	DName	Description
1	Ahmed	M	ahmed@gmail.com	9876786545	1000	Development	Our product development group
2	Peter	M	peter@hotmail.com	NULL	1000	Development	Our product development group
3	Lilly	F	lilly@gmail.com	8798765643	1001	HR	Our human resurece management group
4	Tom	M	tom@gmail.com	9454323454	1001	HR	Our human resurece management group
5	Tom	M	tom_1@gmail.com	9878765434	1001	HR	Our human resurece management group

EmpId	Name	Gender	Email	Contact	Did	Did	DName	Description
1	Ahmed	M	ahmed@gmail.com	9876786545	1000	1000	Development	Our product development group
2	Peter	M	peter@hotmail.com	NULL	1000	1001	HR	Our human resurece management group
3	Lilly	F	lilly@gmail.com	8798765643	1001	1002	QA	Our quality assurance group
4	Tom	M	tom@gmail.com	9454323454	1001			
5	Tom	M	tom_1@gmail.com	9878765434	1001			

Foreign key

- Foreign Key: A foreign key is a “copy” of a primary key that has been exported from one relation into another to represent the existence of a relationship between them.
- A foreign key is a copy of the whole of its parent primary key i.e if the primary key is composite, then so is the foreign key.
- Foreign key values do not (usually) have to be unique.
- Foreign keys can also be null.
- A composite foreign key cannot have some attribute(s) null and others non-null.

Update And Delete Constraints

- What happens if I delete or update a parent record
 - No Action
 - Set Null
 - Set Default
 - Cascade

EmpId	Name	Gender	Email	Contact	Did
1	Ahmed	M	ahmed@gmail.com	9876786545	1000
2	Peter	M	peter@hotmail.com	NULL	1000
3	Lilly	F	lilly@gmail.com	8798765643	1001
4	Tom	M	tom@gmail.com	9454323454	1001
5	Tom	M	tom_1@gmail.com	9878765434	1001

Did	DName	Description
1000	Development	Our product development group
1001	HR	Our human resurece management group
1002	QA	Our quality assurance group

Demo

Let's see a demo on Foreign Key

Database Design Task - 1 [Stock Management System]

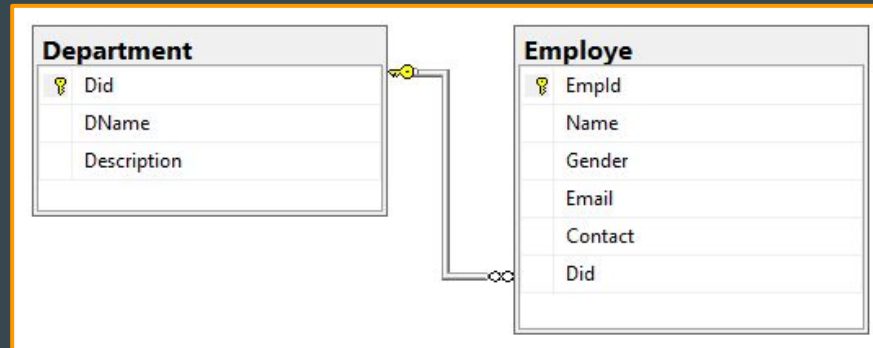
- A book shop contains multiple books.
 - A customer can buy multiple books.
 - An old customer can visit again and again.
 - Books are sold depending on its availability.
-
- **Objects**
 - Book
 - Customer
 - **Relationship**
 - Customer - Book
 - $1 \rightarrow n$
 - $n \rightarrow 1$
 - $n \rightarrow n$ (M:M)

Database Design - 3 Key Rules For Relationships

After identifying the objects and relationship among them apply these three rules to create database structure.

1. One table for each object
2. For 1:M relationship. 1 will become master and M will become child i.e., primary key of 1 will act as a foreign key for M.

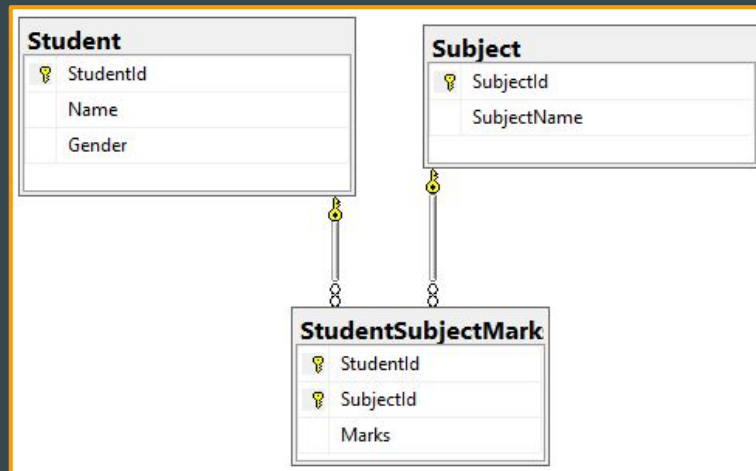
Eg : Department : Employees is 1 : M



Database Design - 3 Key Rules

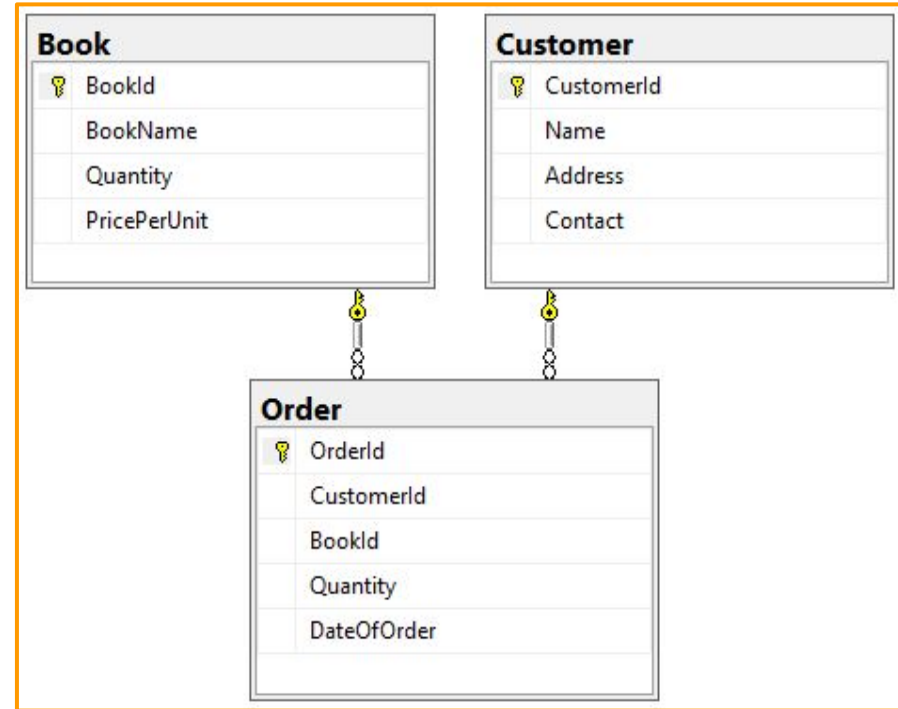
3. M:M relationship. Both the objects will become master and there will be one more new table called as transaction table or child table with primary keys of masters as foreign Keys in it.

Eg : Student : Subject is M : M



Database Design

- Objects
 - Book
 - Customer
- Relationship
 - Customer - Book
 - $1 \rightarrow n$
 - $n \rightarrow 1$
 - $n \rightarrow n$ (M:M)



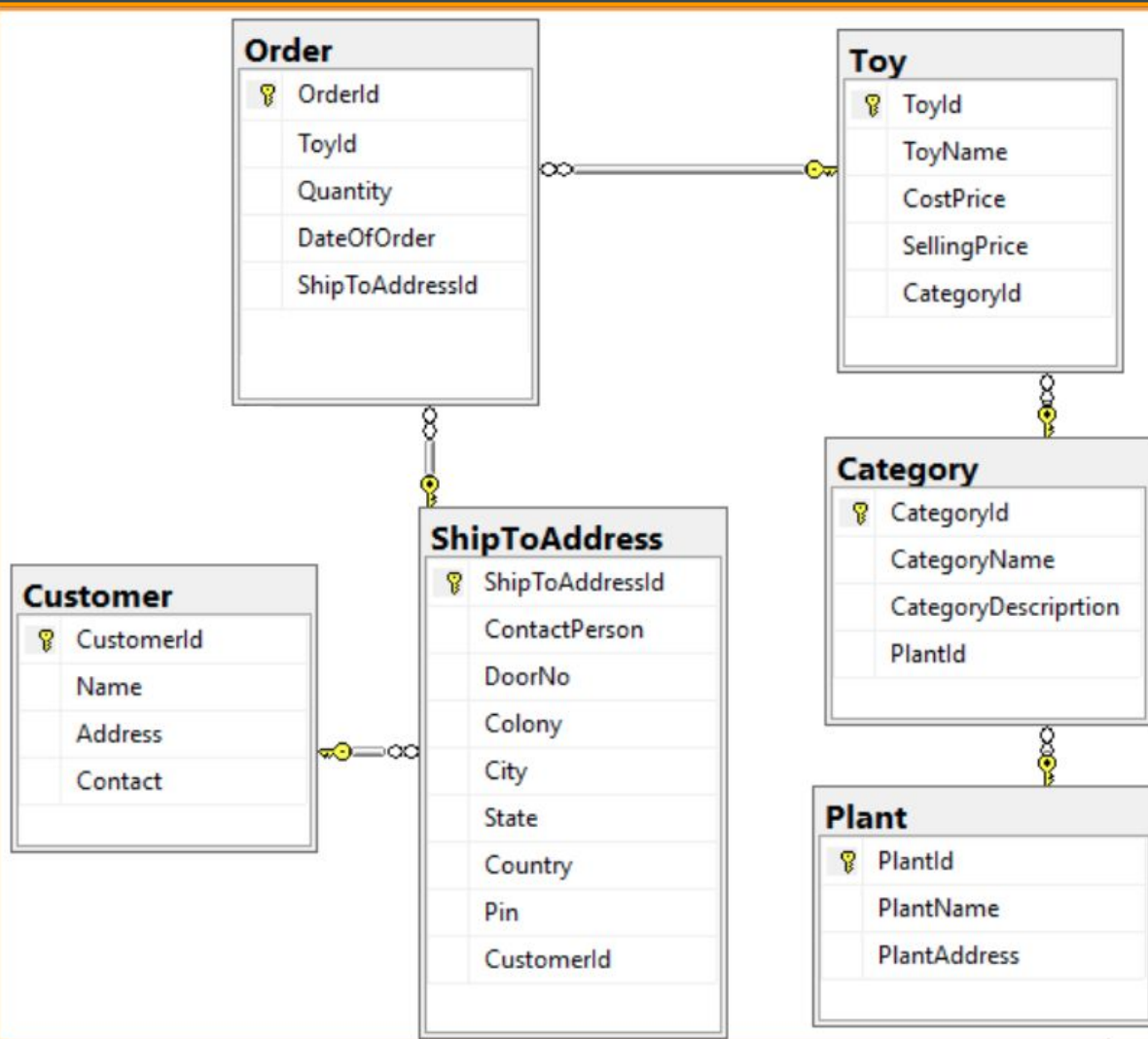
Database Design Task - 2 [Stock Management System]

- A Toy manufacturing company manufactures different types of toys.
- The company has several manufacturing plants.
- Each plant manufactures different types of toys.
- A customer can place order for these toys.
- Each customer has multiple ship-to addresses.

Objects And Relationship

- Objects
 - Plant
 - Category
 - Toy
 - Customer
 - ShipToAddress
- Relationship
 - Plant : Category ($1 \rightarrow n$)(1:M)
 - Category : Toy ($1 \rightarrow n$)(1:M)
 - Customer : ShipToAddress ($1 \rightarrow n$)(1:M)
 - Toy : ShipToAddress ($n \rightarrow n$)
{Order}(M:M)

Database Design



Database Design Task - 3 [Employee Travel History]

Design a portal that maintains information about employees from various organizations travelling on site to various countries.

- **Objects**

- Organization
- Employee
- Country

- **Relationship**

- Organization : Employee $(1 \rightarrow n)(1:M)$
- Employee : Country $(n \rightarrow n)(M:M)$
 - {Travel}

IT Workshop Management System - Scenario

Admin announces a workshop on a topic and assign it to the trainer. New Students Register For the workshop and send a request to attend. Old Students can login and view the upcoming workshops and send a request for it without registration. Admin will approve the request sent by both kind of students and send a confirmation mail along with login credentials (For new students of the portal). Both kind of students need to carry the copy of this mail on the day of workshop as a pass. After the completion of the workshop they will have secured access to the material of the workshop on portal.

IT Workshop Management System - Scenario

Admin can View, Edit and Delete any workshop after announcing. Admin can view, Edit and delete the information about the registered candidates. Can change his own password and can reset the password of any student. Admin can View, Edit and Delete any Trainer. Admin will upload the material in .zip format against a workshop. Admin can View, Edit and Delete any material uploaded against a workshop.

Students can view, Edit and delete their own details and can change their password. Can download the Material. Can view the status of the request for workshop.

Identifying Objects

- Admin
- Trainer
- Student
- Workshop
- Material

Objects & Relationships

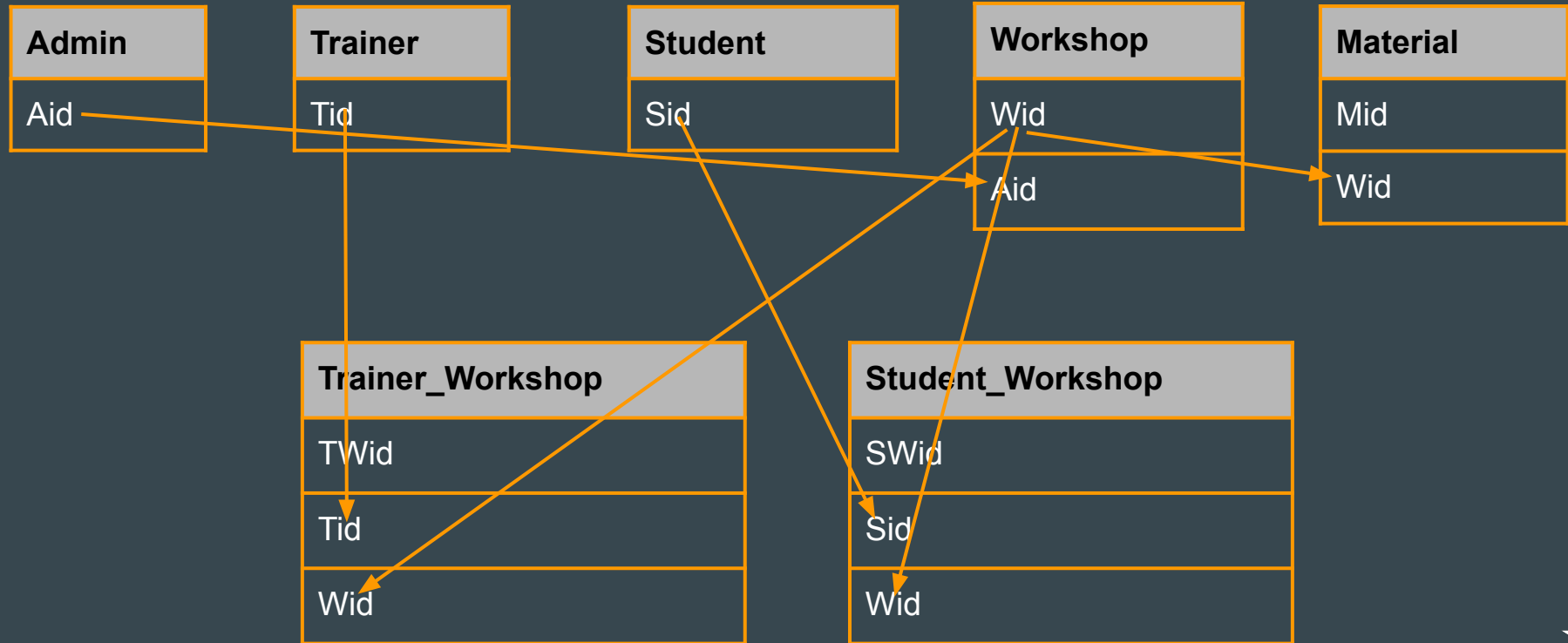
- Objects

- Admin
- Trainer
- Student
- Workshop
- Material

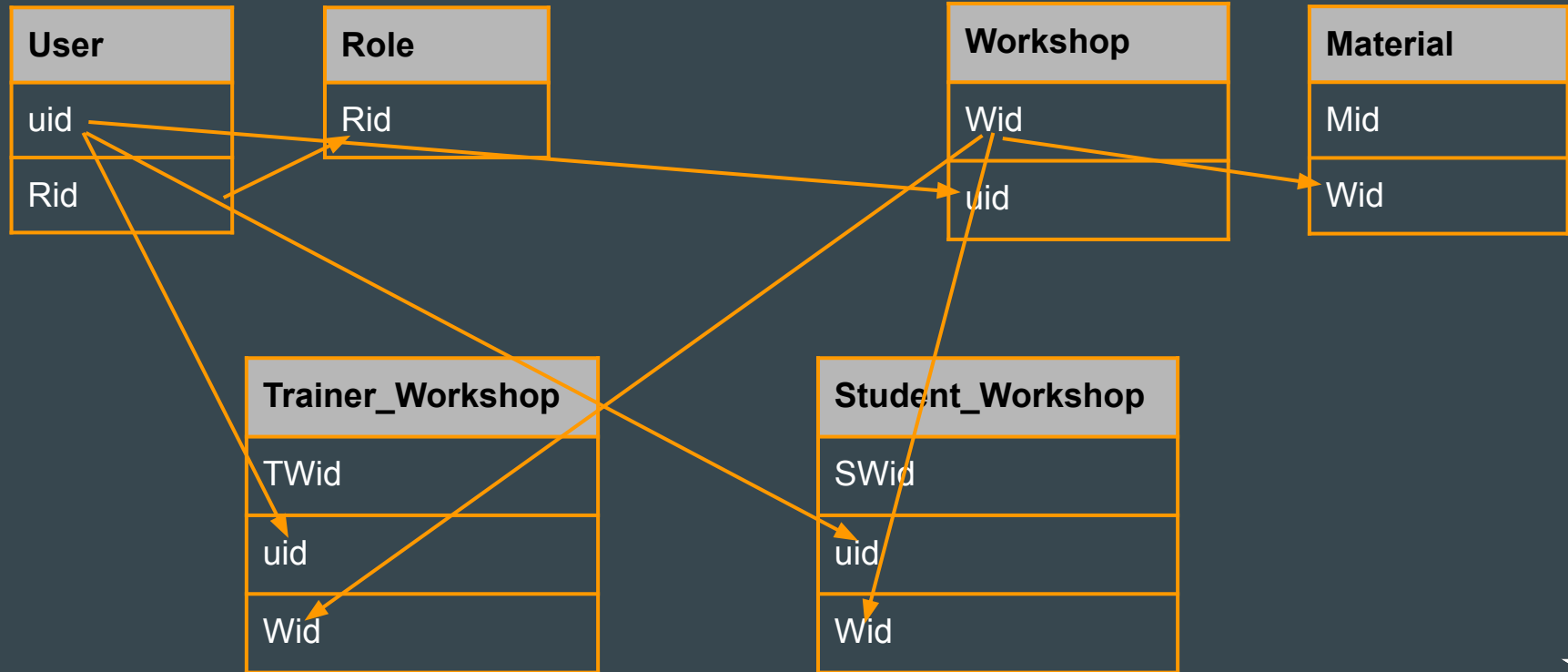
- Relationships

- Admin : Workshop (1:M)
- Trainer : Workshop (M:M)
- Student : Workshop (M:M)
- Material : Workshop (M:1)

Database Design



Database Design



T-SQL (Transact - SQL)

- The database query and programming language used to access data, query data, update data, manage database.
- Two main programming categories:
 - **DDL** : Create, Alter and Drop Objects.
 - **DML** : Select, Insert, Update and Delete operations.

DML - Data Manipulation Language

- Insert
- Update
- Delete
- **Select**

Insert Query On Department

```
insert into Department (Did,DName,Description)
values (1003,'Testing','Our core testing team')
```

Did	DName	Description
1000	Development	Our product development group
1001	HR	Our human resurece management group
1002	QA	Our quality assurance group
1003	Testing	Our core testing team
1004	QC	A team which controls quality

Department	
	Did
	DName
	Description

Demo

Let's see a demo on **Insert**

Delete And Update Query On Department

```
delete from Department where Did=1003
```

```
update Department  
set DName= 'Quality Control', Description='Control Team'  
where Did=1004
```

Department

 Did
DName
Description

Did	DName	Description
1000	Development	Our product development group
1001	HR	Our human resource management group
1002	QA	Our quality assurance group
1004	Quality Control	Control Team

Did	DName	Description
1000	Development	Our product development group
1001	HR	Our human resource management group
1002	QA	Our quality assurance group
1003	Testing	Our core testing team
1004	QC	A team which controls quality

Demo

Let's see a demo on delete and update

Generating And Inserting Dummy Records

- We use this online tool to generate ample number of dummy data to work with different queries.
- <https://www.generatedata.com/>

Demo

Let's see a demo on generating dummy data

Select Query On Employee Table

- Select * From <Table Name>, Table & Column Aliasing, Distinct
- Expressions on columns : Mathematical & Cases
- Order By
 - Single & Multiple Column
 - Top, Top With Ties
- Where
 - =,<>,<,>,<=,>= (conditions)
 - In, Not In, Between, Not Between, Is null, Is not null
 - Like → %,[]
 - soundex()

Demo

Let's see a demo on `select query`

Scalar Functions

- A scalar valued function is a function that takes one or more values but returns a single value.
- **String Functions** : Len(), Lower(), Ltrim(), Substring(), String_Split()
- **Other Functions** : Cast(), Convert(), Isnull()
- **Mathematical Functions** : Ceiling(), Floor(), Round(), Rand()
- **Aggregate Functions** : Count(), Sum(), Avg(), Max(), Min()
- **DateTime Functions** : getdate(), year(), month(), day(), DateAdd(), Datediff(), Datepart(), DateName()

Demo

Let's see a demo on **Scalar Functions**

Grouping Result Set

```
select Distinct Gender From Employee
select count(*) as Females from Employee Where Gender='Female'
select count(*) as Males from Employee Where Gender='Male'
```

Gender	NoOfEmps
Female	54
Male	46

```
select Gender, count(*) as NoOfEmps From Employee
```

```
select Gender, count(*) as NoOfEmps From Employee
```

%

Messages

```
Msg 8120, Level 16, State 1, Line 183
Column 'Employee.Gender' is invalid in the select list
because it is not contained in either an aggregate function
or the GROUP BY clause.
```

```
select Gender, count(*) as NoOfEmps From Employee
group by Gender
```

Gender
Female
Male
Females
54
Males
46

Grouping Result Set

- Group By
- Having
- FWGHO
 - The FROM clause assembles the data from the data sources.
 - The WHERE clause restricts the rows based on the conditions.
 - The GROUP BY clause assembles subsets of data.
 - Aggregate functions are calculated.
 - The HAVING clause filters the subsets of data.
 - Any remaining expressions are calculated.
 - The ORDER BY sorts the results.

Demo

Let's see a demo on Group By

Joins (Multiplication)

- **Joins** : Used to extract data from multiple tables.
 - Inner Join
 - Outer Join
 - Left Outer Join
 - Right Outer Join
 - Full Outer Join

Inner Join

Get only the matched rows from left(child) and the right(master) table.

EmpId	Name	Gender	Email	Contact	Did
1	Ahmed	M	ahmed@gmail.com	9876786545	1000
2	Peter	M	peter@hotmail.com	NULL	1000
3	Lilly	F	lilly@gmail.com	8798765643	1001
4	Tom	M	tom@gmail.com	9454323454	1001
5	Tom	M	tom_1@gmail.com	9878765434	1001
6	Jack	M	jack@gmail.com	9876765453	NULL
7	Bob	M	bob@gmail.com	9878765654	NULL

Did	DName	Description
1000	Development	Our product development group
1001	HR	Our human resurece management group
1002	QA	Our quality assurance group

EmpId	Name	Gender	Email	Contact	Did	DName	Description
1	Ahmed	M	ahmed@gmail.com	9876786545	1000	Development	Our product development group
2	Peter	M	peter@hotmail.com	NULL	1000	Development	Our product development group
3	Lilly	F	lilly@gmail.com	8798765643	1001	HR	Our human resurece management group
4	Tom	M	tom@gmail.com	9454323454	1001	HR	Our human resurece management group
5	Tom	M	tom_1@gmail.com	9878765434	1001	HR	Our human resurece management group

Left Outer Join

Get all the rows from left(child) and matched rows from the right(master) table.

Empld	Name	Gender	Email	Contact	Did
1	Ahmed	M	ahmed@gmail.com	9876786545	1000
2	Peter	M	peter@hotmail.com	NULL	1000
3	Lilly	F	lilly@gmail.com	8798765643	1001
4	Tom	M	tom@gmail.com	9454323454	1001
5	Tom	M	tom_1@gmail.com	9878765434	1001
6	Jack	M	jack@gmail.com	9876765453	NULL
7	Bob	M	bob@gmail.com	9878765654	NULL

Did	DName	Description
1000	Development	Our product development group
1001	HR	Our human resurece management group
1002	QA	Our quality assurance group

Empld	Name	Gender	Email	Contact	Did	DName	Description
1	Ahmed	M	ahmed@gmail.com	9876786545	1000	Development	Our product development group
2	Peter	M	peter@hotmail.com	NULL	1000	Development	Our product development group
3	Lilly	F	lilly@gmail.com	8798765643	1001	HR	Our human resurece management group
4	Tom	M	tom@gmail.com	9454323454	1001	HR	Our human resurece management group
5	Tom	M	tom_1@gmail.com	9878765434	1001	HR	Our human resurece management group
6	Jack	M	jack@gmail.com	9876765453	NULL	NULL	NULL
7	Bob	M	bob@gmail.com	9878765654	NULL	NULL	NULL

Right Outer Join

Get only the matched rows from left(child) and all rows from the right(master) table.

Empld	Name	Gender	Email	Contact	Did
1	Ahmed	M	ahmed@gmail.com	9876786545	1000
2	Peter	M	peter@hotmail.com	NULL	1000
3	Lilly	F	lilly@gmail.com	8798765643	1001
4	Tom	M	tom@gmail.com	9454323454	1001
5	Tom	M	tom_1@gmail.com	9878765434	1001
6	Jack	M	jack@gmail.com	9876765453	NULL
7	Bob	M	bob@gmail.com	9878765654	NULL

Did	DName	Description
1000	Development	Our product development group
1001	HR	Our human resurece management group
1002	QA	Our quality assurance group

Empld	Name	Gender	Email	Contact	Did	DName	Description
1	Ahmed	M	ahmed@gmail.com	9876786545	1000	Development	Our product development group
2	Peter	M	peter@hotmail.com	NULL	1000	Development	Our product development group
3	Lilly	F	lilly@gmail.com	8798765643	1001	HR	Our human resurece management group
4	Tom	M	tom@gmail.com	9454323454	1001	HR	Our human resurece management group
5	Tom	M	tom_1@gmail.com	9878765434	1001	HR	Our human resurece management group
NULL	NULL	NULL	NULL	NULL	1002	QA	Our quality assurance group

Full Outer Join

Get all the rows from left(child) and the right(master) table.

Empld	Name	Gender	Email	Contact	Did
1	Ahmed	M	ahmed@gmail.com	9876786545	1000
2	Peter	M	peter@hotmail.com	NULL	1000
3	Lilly	F	lilly@gmail.com	8798765643	1001
4	Tom	M	tom@gmail.com	9454323454	1001
5	Tom	M	tom_1@gmail.com	9878765434	1001
6	Jack	M	jack@gmail.com	9876765453	NULL
7	Bob	M	bob@gmail.com	9878765654	NULL

Did	DName	Description
1000	Development	Our product development group
1001	HR	Our human resurece management group
1002	QA	Our quality assurance group

Empld	Name	Gender	Email	Contact	Did	DName	Description
1	Ahmed	M	ahmed@gmail.com	9876786545	1000	Development	Our product development group
2	Peter	M	peter@hotmail.com	NULL	1000	Development	Our product development group
3	Lilly	F	lilly@gmail.com	8798765643	1001	HR	Our human resurece management group
4	Tom	M	tom@gmail.com	9454323454	1001	HR	Our human resurece management group
5	Tom	M	tom_1@gmail.com	9878765434	1001	HR	Our human resurece management group
6	Jack	M	jack@gmail.com	9876765453	NULL	NULL	NULL
7	Bob	M	bob@gmail.com	9878765654	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	1002	QA	Our quality assurance group

Writing Join Query

EmpId	Name	Gender	Email	Contact	Did
1	Ahmed	M	ahmed@gmail.com	9876786545	1000
2	Peter	M	peter@hotmail.com	NULL	1000
3	Lilly	F	lilly@gmail.com	8798765643	1001
4	Tom	M	tom@gmail.com	9454323454	1001
5	Tom	M	tom_1@gmail.com	9878765434	1001
6	Jack	M	jack@gmail.com	9876765453	NULL
7	Bob	M	bob@gmail.com	9878765654	NULL

Did	DName	Description
1000	Development	Our product development group
1001	HR	Our human resurece management group
1002	QA	Our quality assurance group

```
SELECT Employee.EmpId, Employee.Name, Employee.Gender, Employee.Email, Employee.Contact,
Department.Did, Department.DName, Department.Description
FROM Employee full outer JOIN Department ON Employee.Did = Department.Did
```

inner join or left outer join or right outer join

EmpId	Name	Gender	Email	Contact	Did	DName	Description
1	Ahmed	M	ahmed@gmail.com	9876786545	1000	Development	Our product development group
2	Peter	M	peter@hotmail.com	NULL	1000	Development	Our product development group
3	Lilly	F	lilly@gmail.com	8798765643	1001	HR	Our human resurece management group
4	Tom	M	tom@gmail.com	9454323454	1001	HR	Our human resurece management group
5	Tom	M	tom_1@gmail.com	9878765434	1001	HR	Our human resurece management group
6	Jack	M	jack@gmail.com	9876765453	NULL	NULL	NULL
7	Bob	M	bob@gmail.com	9878765654	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	1002	QA	Our quality assurance group

Demo

Let's see a demo on Joins

Joining Three Tables

StudentId	Name	Gender
1	David	M
2	Adam	M
3	Lilly	F
4	Catrin	F
5	Paul	M

SubjectId	SubjectName
101	Maths
102	Physics
103	Chemistry
104	Social

StudentId	SubjectId	Marks
1	101	67
1	102	68
1	103	56
2	101	78
2	102	77
2	103	79
3	101	NULL
3	102	56
3	103	55

StudentId	Name	SubjectName	Marks
1	David	Maths	67
1	David	Physics	68
1	David	Chemistry	56
2	Adam	Maths	78
2	Adam	Physics	77
2	Adam	Chemistry	79
3	Lilly	Maths	NULL
3	Lilly	Physics	56
3	Lilly	Chemistry	55

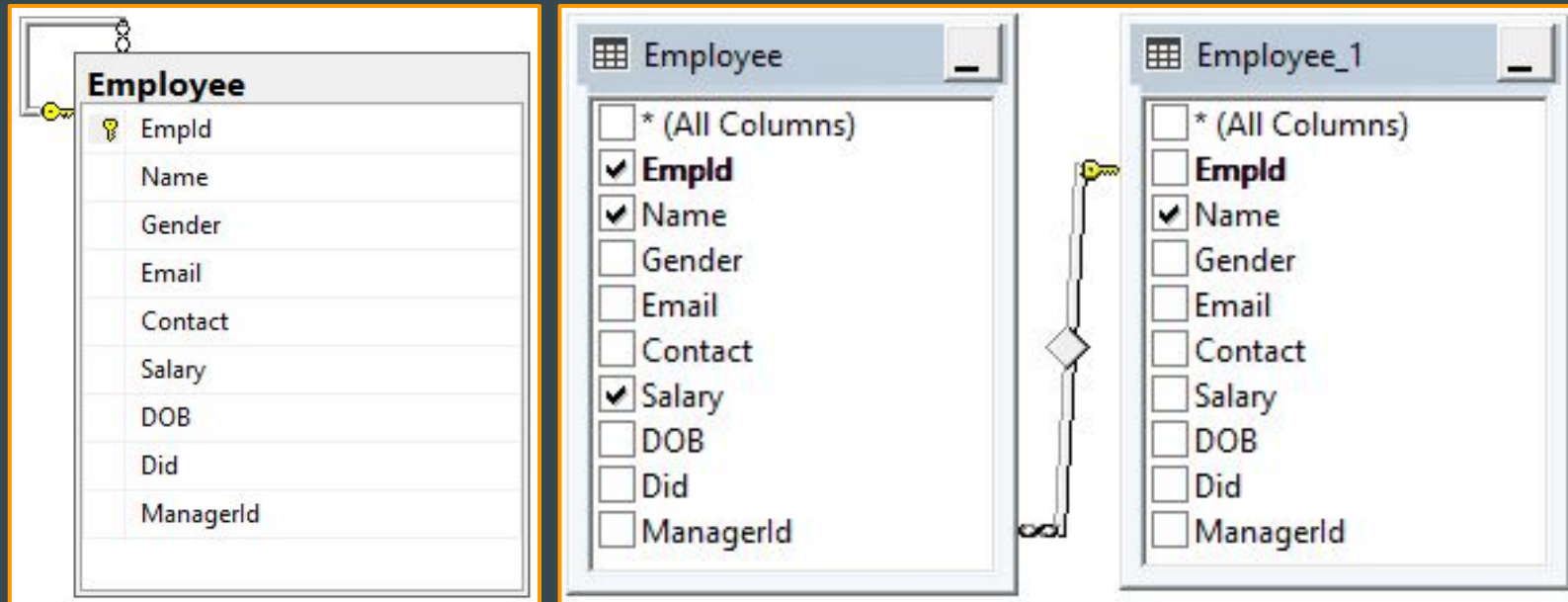
```
SELECT Student.StudentId, Student.Name, Subject.SubjectName, StudentSubjectMarks.Marks
FROM StudentSubjectMarks INNER JOIN
      Student ON StudentSubjectMarks.StudentId = Student.StudentId INNER JOIN
      Subject ON StudentSubjectMarks.SubjectId = Subject.SubjectId
```

Demo

Let's see a demo on **Joining three tables**

Self Join

- Joining a table with itself



```
SELECT Employee.EmpId, Employee.Name, Employee.Salary, Employee_1.Name AS Manager
FROM Employee INNER JOIN Employee AS Employee_1 ON Employee.ManagerId = Employee_1.EmpId
```



Demo

Let's see a demo on Self Join

Union (Addition)

- **Union** : Used to append records of more than one table
 - Union All ($A \cup B$ with duplicate records)
 - Union ($A \cup B$ without duplicate records)
 - Intersect ($A \cap B$)
 - Except ($A - B$)

Union ALL

CustomerBusinessA

CustomerId	Name	Address	Contact
1	Jack	US	9878765434
2	Peter	UK	9878765454
3	Tom	UK	5676545432
4	Lilly	SA	9878765453
5	Ahmed	India	9587676543

CustomerBusinessB

CustomerId	Name	Address	Contact
1000	Tom	UK	5676545432
1001	Lilly	SA	9878765453
1002	Ahmed	India	9587676543
1003	Bob	Canada	9897123432
1004	Adam	Australia	8787654543

```
select Name,Address,Contact from CustomerBusinessA
Union All
```

```
select Name,Address,Contact from CustomerBusinessB
```

Name	Address	Contact
Jack	US	9878765434
Peter	UK	9878765454
Tom	UK	5676545432
Lilly	SA	9878765453
Ahmed	India	9587676543
Tom	UK	5676545432
Lilly	SA	9878765453
Ahmed	India	9587676543
Bob	Canada	9897123432
Adam	Australia	8787654543

Union

CustomerBusinessA

CustomerId	Name	Address	Contact
1	Jack	US	9878765434
2	Peter	UK	9878765454
3	Tom	UK	5676545432
4	Lilly	SA	9878765453
5	Ahmed	India	9587676543

CustomerBusinessB

CustomerId	Name	Address	Contact
1000	Tom	UK	5676545432
1001	Lilly	SA	9878765453
1002	Ahmed	India	9587676543
1003	Bob	Canada	9897123432
1004	Adam	Australia	8787654543

```
select Name,Address,Contact from CustomerBusinessA
Union
select Name,Address,Contact from CustomerBusinessB
```

Name	Address	Contact
Adam	Australia	8787654543
Ahmed	India	9587676543
Bob	Canada	9897123432
Jack	US	9878765434
Lilly	SA	9878765453
Peter	UK	9878765454
Tom	UK	5676545432

Intersect

CustomerBusinessA

CustomerId	Name	Address	Contact
1	Jack	US	9878765434
2	Peter	UK	9878765454
3	Tom	UK	5676545432
4	Lilly	SA	9878765453
5	Ahmed	India	9587676543

CustomerBusinessB

CustomerId	Name	Address	Contact
1000	Tom	UK	5676545432
1001	Lilly	SA	9878765453
1002	Ahmed	India	9587676543
1003	Bob	Canada	9897123432
1004	Adam	Australia	8787654543

```
select Name,Address,Contact from CustomerBusinessA
intersect
select Name,Address,Contact from CustomerBusinessB
```

Name	Address	Contact
Ahmed	India	9587676543
Lilly	SA	9878765453
Tom	UK	5676545432

Except

CustomerBusinessA

CustomerId	Name	Address	Contact
1	Jack	US	9878765434
2	Peter	UK	9878765454
3	Tom	UK	5676545432
4	Lilly	SA	9878765453
5	Ahmed	India	9587676543

```
select Name,Address,Contact from CustomerBusinessA
Except
select Name,Address,Contact from CustomerBusinessB
```

Name	Address	Contact
Jack	US	9878765434
Peter	UK	9878765454

CustomerBusinessB

CustomerId	Name	Address	Contact
1000	Tom	UK	5676545432
1001	Lilly	SA	9878765453
1002	Ahmed	India	9587676543
1003	Bob	Canada	9897123432
1004	Adam	Australia	8787654543

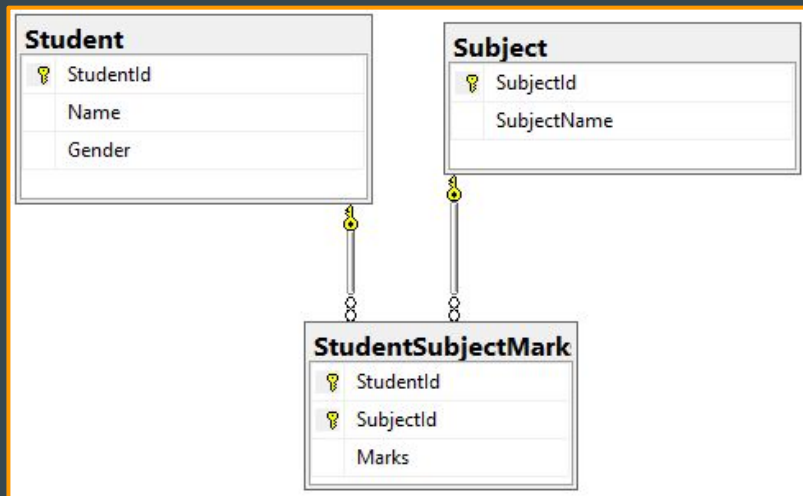
Demo

Let's see a demo on Union

Sub Queries

- Subquery or Inner query or Nested query is a query inside a query.
- **Simple Subqueries** : Outer query is executed based upon the results of inner query.
 - Scalar Subqueries
 - Multivalued Subqueries
- **Correlated Subquery** :
 - It is a sub-query that uses values from the outer query.
 - In this case the inner query has to be executed for every row of outer query.
- **Common Table Expression (CTE) (Alternative To Views)** :
 - It defines the subquery at the beginning of the query using the WITH command called as CTE
 - CTE can then be accessed multiple times below CTE in main query.

Scenario - Simple Subquery



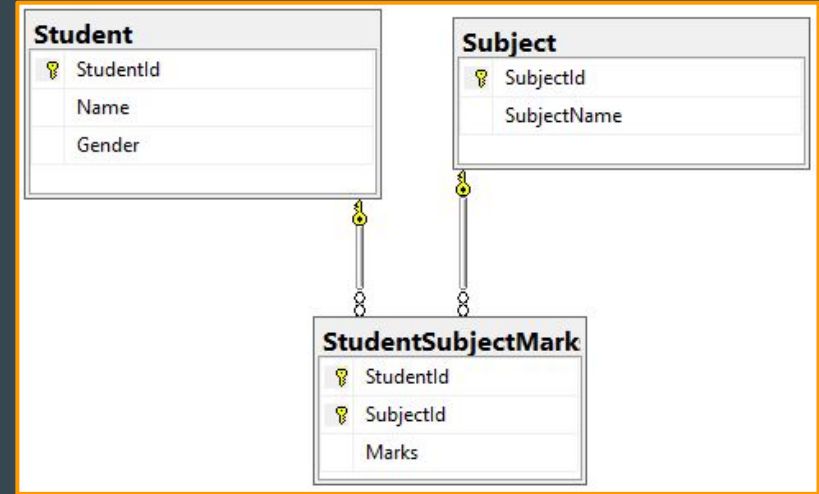
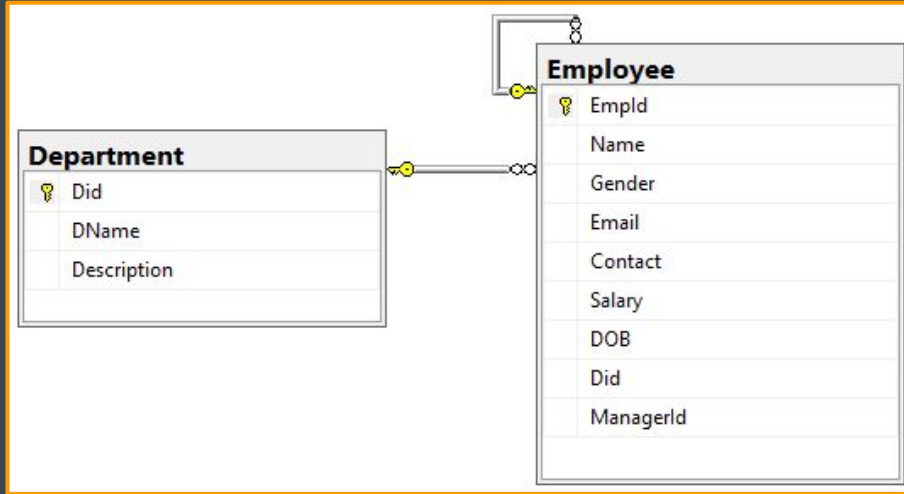
- Get the subject name which has got the highest marks. (Simple Scalar Subquery)
- Get the students who are taking **Maths** exam (Nested Scalar Subquery)
- Get the students who has not taken at least one exam (Multivalued Subquery)

Note: A LEFT [OUTER] JOIN can be faster than an equivalent subquery.

Demo

Let's see a demo on Simple SubQueries

Scenario - Correlated Subquery



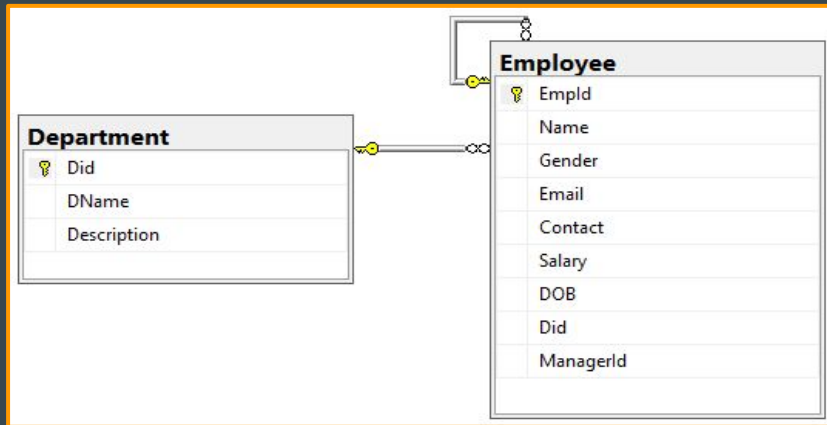
- Get the employee details with lowest salary from every department.
- Get the topper student details from every subject.

Demo

Let's see a demo on **Correlated SubQueries**

Scenario - Common Table Expression Subquery

Common Table Expression (CTE) (Alternative To Views) : It defines the subquery at the beginning of the query using the WITH command called as CTE. It can then be accessed multiple times below CTE in main query



Department	Details	Result Value
Development	No Of Employees	21
Development	Sum Of Salaries	142456
HR	No Of Employees	18
HR	Sum Of Salaries	143777
QA	No Of Employees	11
QA	Sum Of Salaries	85690
Quality Control	No Of Employees	26
Quality Control	Sum Of Salaries	188492
Testing	No Of Employees	24
Testing	Sum Of Salaries	182235

- Get department name and number of employees in each dept along with department name and total salary being paid to every department

A Simple CTE

```
WITH Departments_CTE (DName, EmpId, Salary)
AS
-- Define the CTE query.
(SELECT Department.DName, Employee.EmpId, Employee.Salary
FROM Employee FULL OUTER JOIN Department ON Employee.Did = Department.Did
)
-- Define the outer query referencing the CTE name.
SELECT DName as Department, 'No Of Employees' as Details, Count(*) as ResultValue
FROM Departments_CTE
GROUP BY DName
Union
SELECT DName as Department, 'Sum Of Salaries' as Details, Sum(Salary) as ResultValue
FROM Departments_CTE
GROUP BY DName
Order by DName
```

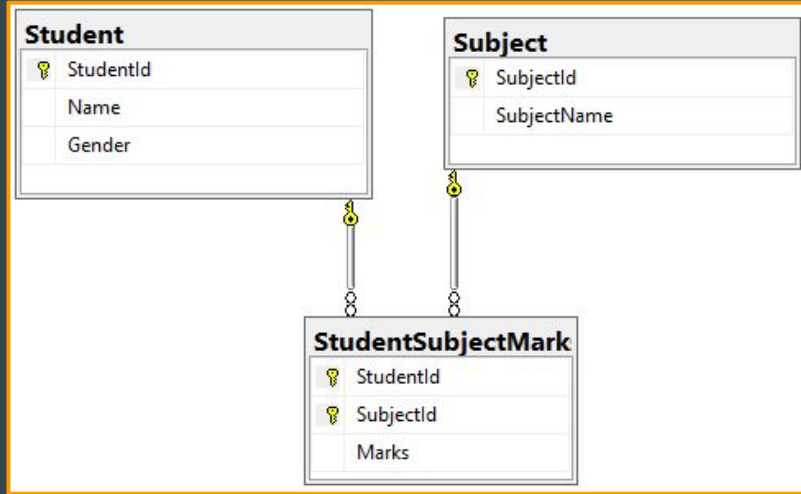
Demo

Let's see a demo on **CTE SubQueries**

Views

- A view is a list of columns or a series of records retrieved from one or more existing tables, or as a combination of one or more views and one or more tables.
- Use views to denormalize or simplify complex joins and hide any keys used to link data within the database schema.
- In other words it is use full for generating various reports easily.

Senario - Views



- Get the subject name and number of students taking exam on that subject.
- Get the student name, subject name and marks they get in that subject.
- Get the student name and total mark they got.

Demo

Let's see a demo for **views**

DML On Views

- You can perform **all operation** on a view that is based **on one table**, that would be entered on the base table.
- You can perform only **select & update operations** on a view which is based on **multiple table**, that would affect the base tables.

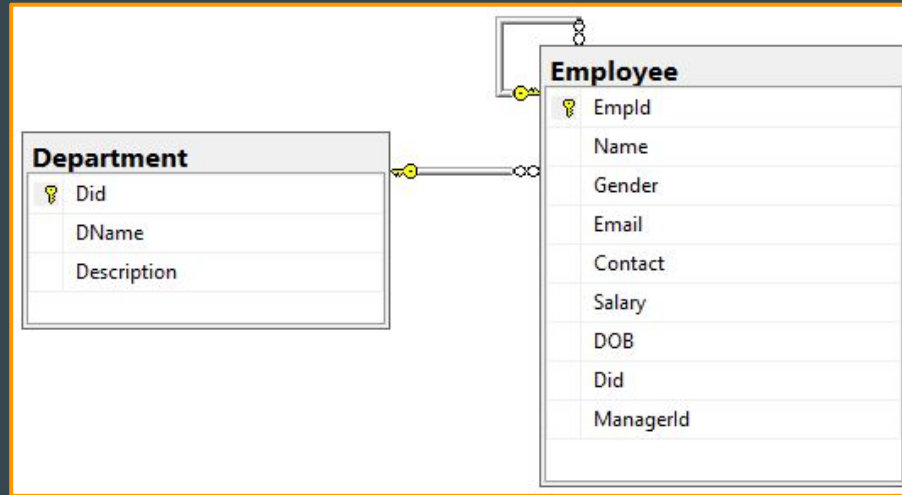
Demo

Let's see a demo for **DML on views**

Stored Procedures

- Stored Procedure is precompiled set of one or more SQL statements which performs some specific task.
- Store Procedure should be executed stand alone using EXEC.
- Store Procedure can return multiple parameters.
- Store Procedure can be used to implement transaction.
- Create, Alter and Drop Stored Procedures
 - With No Parameters and No Return values
 - With Parameters and No Return values
 - With Parameters and Return values

Scenario - Stored Procedure

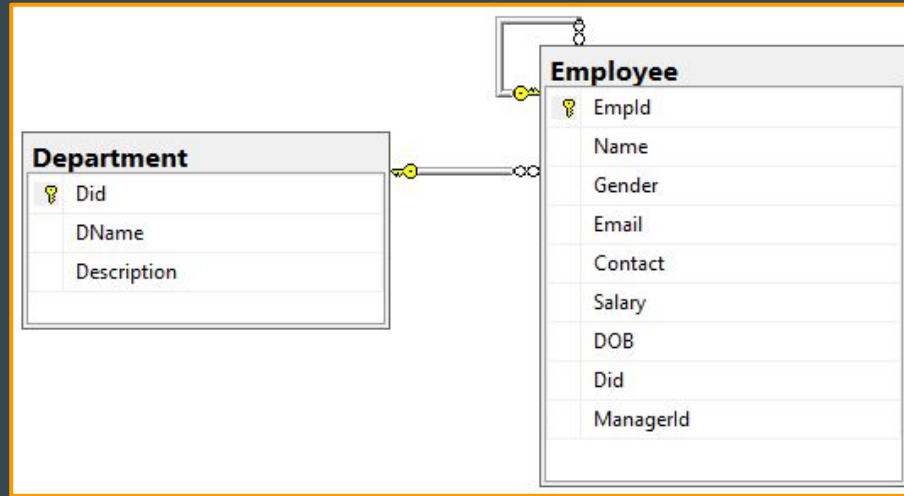


- Get all the employees with department details.
- Get all the employees with department details of a particular department.
- Get the salary of a particular employee.

Demo

Let's see a demo for **Stored Procedures**

Scenario - Stored Procedure With If Else



- Get all the employees with department details when department Id is not passed or NULL else all the employees with department details of a particular department.

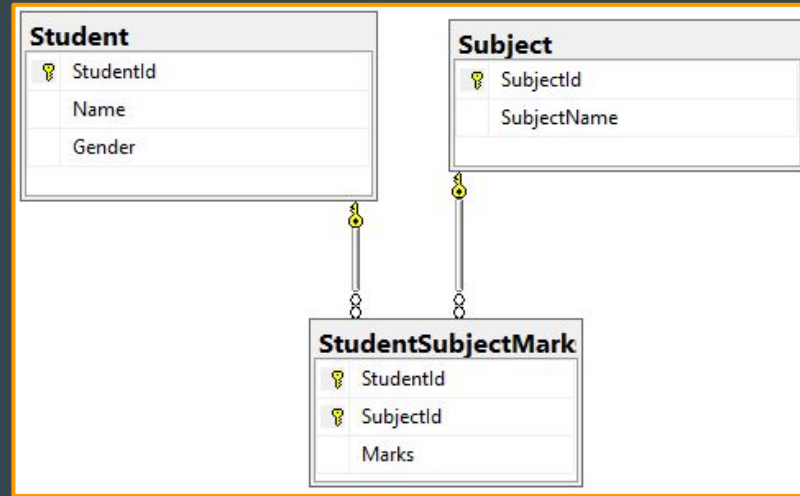
Demo

Let's see a demo for **Stored Procedures**

Functions

- A Functions is a section of code that performs an independent task.
- Function cannot be executed stand alone, it is always used in a query like with select clause.
- Functions can return one and only one parameter.
- We cannot implement transaction using functions.

Senario - Function



- Get the student name, total mark they got along with grade. Where grade is the value returned from GetGrade() function.

Demo

Let's see a demo for **functions**

Transactions

- **Transaction** is a set of operation (SQL Queries) which must be executed as a group.
- If at all a single operation fails then all the previous operation of that group must be rolled back i.e., undone.

Scenario - Transaction

CustomerId	Name	Balance
12346	Jack	7000
12347	Lilly	9000
12348	Tom	8000
12349	Adam	4000

- Write a transaction to transfer 2000 from Jack's Account to Lilly's Account

Slowly Changing Dimensions

- Dimensions that change slowly over time, rather than changing on regular schedule, time-base.
- In Data Warehouse there is a need to track changes in dimension attributes in order to report historical data.
- Eg: **Salary of employee**, City of a customer, Class of a student, etc.,

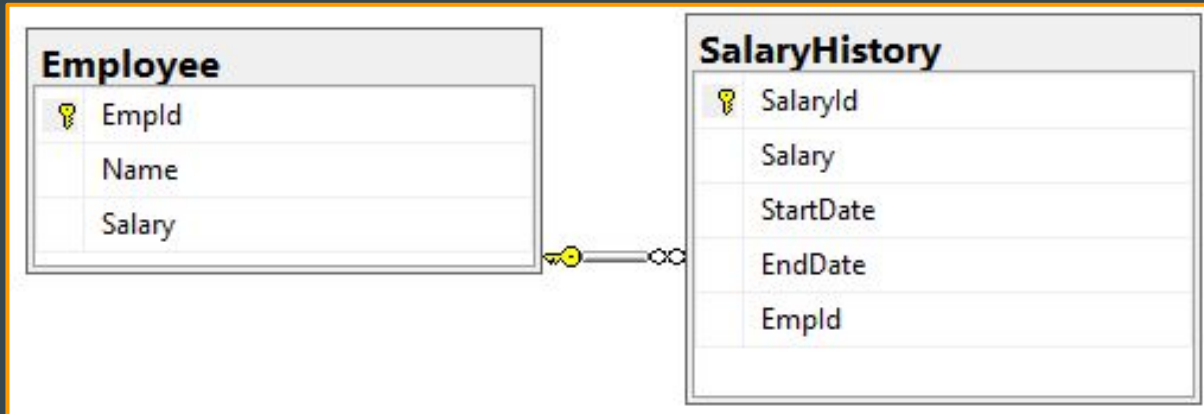
SCD - Senario Salary History

EmpId	Name	Salary		
1	Manzoor	15000		

SalaryId	salary	StartDate	EndDate	EmpId
1000	15000	01 Apr 2017	NULL	1

EmpId	Name	Salary		
1	Manzoor	25000		

SalaryId	salary	StartDate	EndDate	EmpId
1000	15000	01 Apr 2017	01 Apr 2018	1
1001	25000	01 Apr 2018	NULL	1



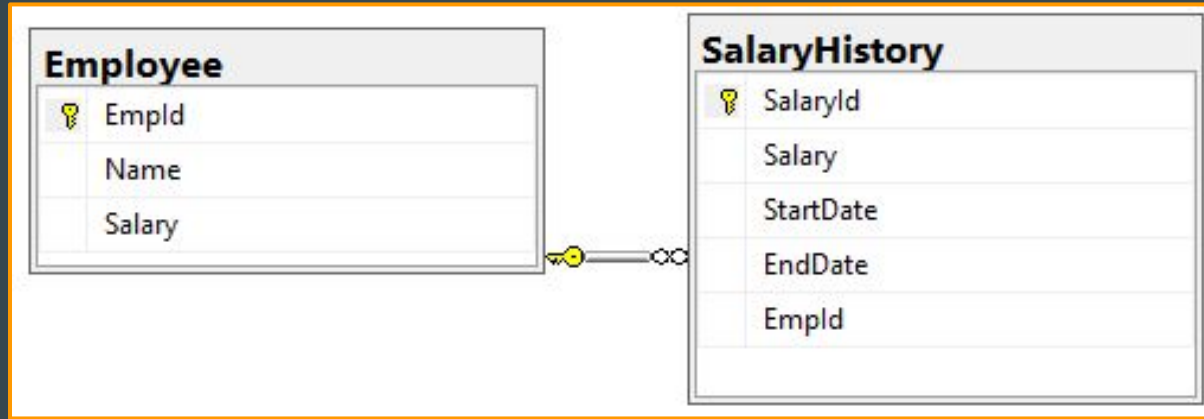
Scope_Identity()

- How to get the EmpId value to insert in SalaryHistory table?
- As EmpId is identity column.
- **Scope_Identity** function returns the recent generated identity value (that is auto incremented values) for that particular scope i.e., user.

Demo

Let's see a demo for `ScopeIdentity()`

Scenario - Transaction



- What if insertion in SalaryHistory Fails?
- Write a transaction to maintain employee salary history.
 - Insert.
 - update.

Demo

Let's see a demo for Transactions

Triggers

- A trigger is an action that is performed behind-the-scenes when an insert, update or delete operation has been performed on a table.
- The trigger must be created in connection to a table.
- Triggers are of 2 types in SQL Server:
- DML Triggers
 - AFTER Triggers
 - INSTEAD OF Triggers
- DDL Triggers

Creating Triggers

```
Create Trigger MyInsteadOfUpdateTrigger
on BankCustomer
instead Of Update
as
print 'Your update did not happened'
```

```
Create Trigger MyAfterDeleteTrigger
on BankCustomer
After Delete
as
print 'Your really deleted a record'
```

Demo

Let's see a demo for Triggers

Insert Operation

Department

DID	Name	Description
100	Testing	Testing Dept
101	Admin	Admin Dept
102	HR	HR Dept
103	Development	Dev Dept

↑
Insert a record

Inserted Table

DID	Name	Description
103	Development	Dev Dept

Deleted Table

DID	Name	Description
-----	------	-------------

Demo

Let's see a demo for **Inserted Table**

Update Operation

Department

DID	Name	Description
100	Testing	Testing Dept
101	Admin	Admin Dept
102	HR	HR Dept
103	Dev	Dev Dept

Update a record

Inserted Table

DID	Name	Description
103	Dev	Dev Dept

Deleted Table

DID	Name	Description
103	Development	Dev Dept

Demo

Let's see a demo for Updated Table

Delete Operation

Department

DID	Name	Description
100	Testing	Testing Dept
101	Admin	Admin Dept
102	HR	HR Dept

Delete a record

Inserted Table

DID	Name	Description
-----	------	-------------

Deleted Table

DID	Name	Description
103	Dev	Dev Dept

Demo

Let's see a demo for Deleted Table

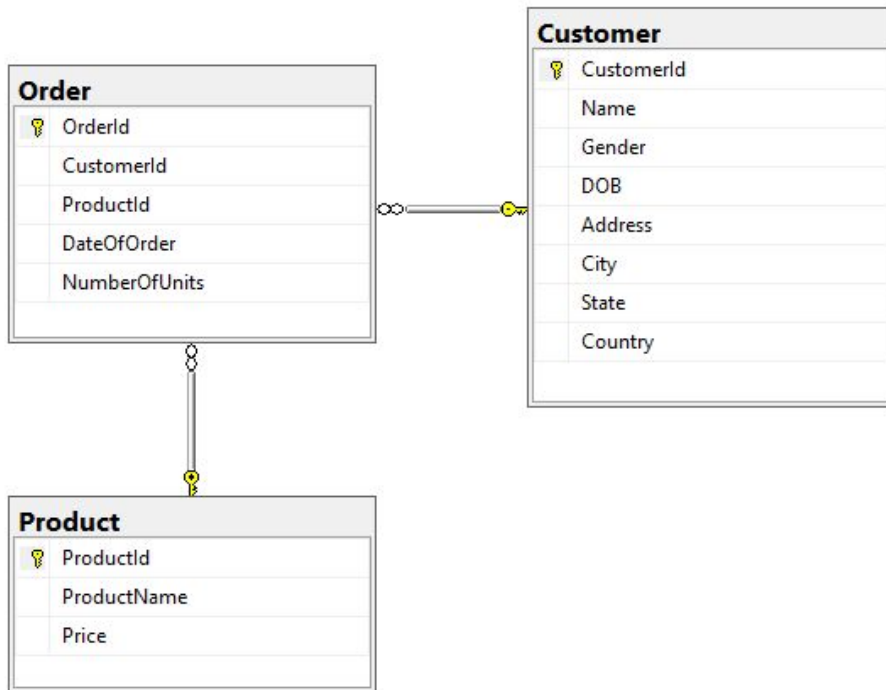
Some Admin Tasks

- Database Backup
- Database Restore
- Generate DB Script

Demo

Let's see a demo on **Admin Tasks**

Assignment



- Write a query which takes the OrderId as input and displays all the customer details and product details along with the last date of order delivery.
- Write a query to display the details of a customer with highest number of orders.
- Write a query to display the business generated from each country.

Note : Product should be delivered within 10 days of order placed.

Thanks

Ask any questions if you have...